

Universidad Autónoma de Zacatecas
Ingeniería de software
Análisis y Diseño de Algoritmos
Proyecto No.1
Algoritmos de Ordenamiento
Dominic Martin Campos Cardona



Algoritmos/Metodos de Ordenamiento

Introducción:

- Los algoritmos de ordenamiento pueden ser descritos como un conjunto de instrucciones que permiten ordenar un conjunto de datos según lo que se pida los siguientes son algunos ejemplo de estos algoritmos de ordenamiento

Bubble Sort:

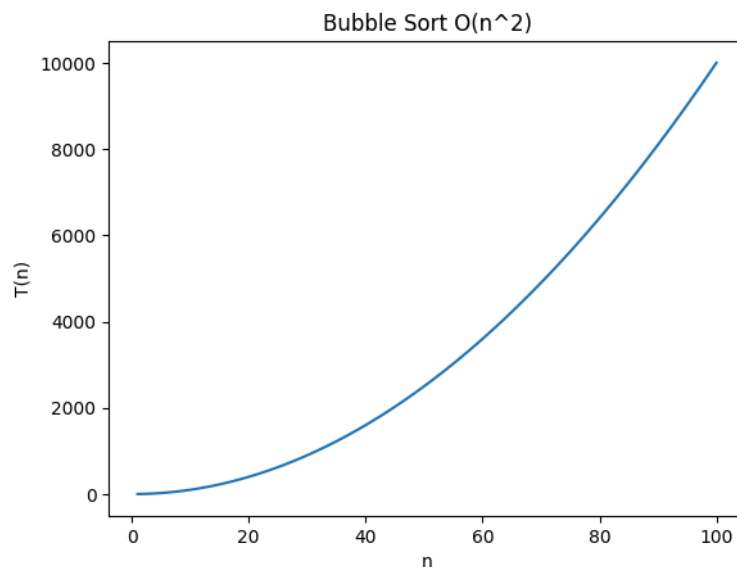
Complejidad:

- Mejor Caso: $O(n)$
- Peor Caso: $O(n^2)$

Observaciones:

- Compara elementos adyacentes para intercambiarlos en caso de que estén en la posición incorrecta
- Es un algoritmo que si bien es simple debido a como hace comparaciones también es poco efectivo

Gráfica:



Selection Sort:

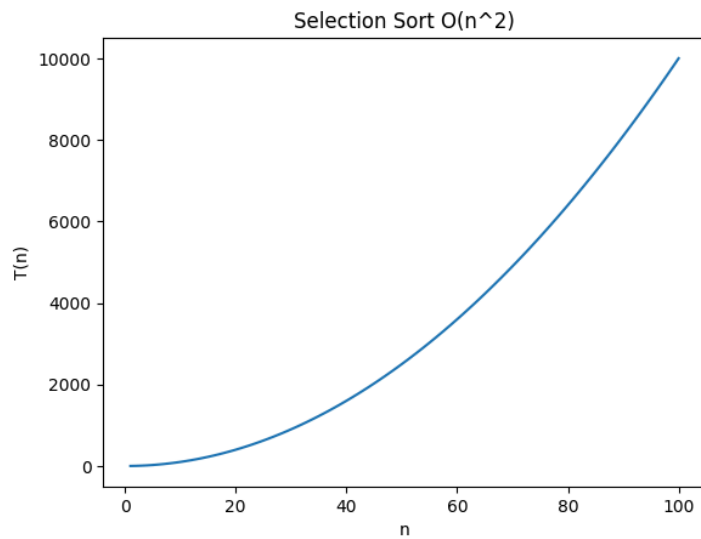
Complejidad:

- Mejor Caso: $O(n^2)$
- Peor Caso: $O(n^2)$

Observaciones:

- En cada iteración busca el elemento mínimo y lo coloca en la posición correcta
- Ya que siempre inicia buscando el más bajo su efectividad no mejora con un conjunto que ya está ordenado
- Por lo anterior es poco eficiente

Gráfica:



Insertion Sort:

Complejidad:

- Mejor Caso: $O(n)$
- Peor Caso: $O(n^2)$

Observaciones:

- El insertion sort como el nombre indica inserta cada elemento ya en la posición correcta después de compararlos con los anteriores
- Por lo anterior es altamente eficiente en arreglos pequeños o casi ordenados

Gráfica:



Merge Sort:

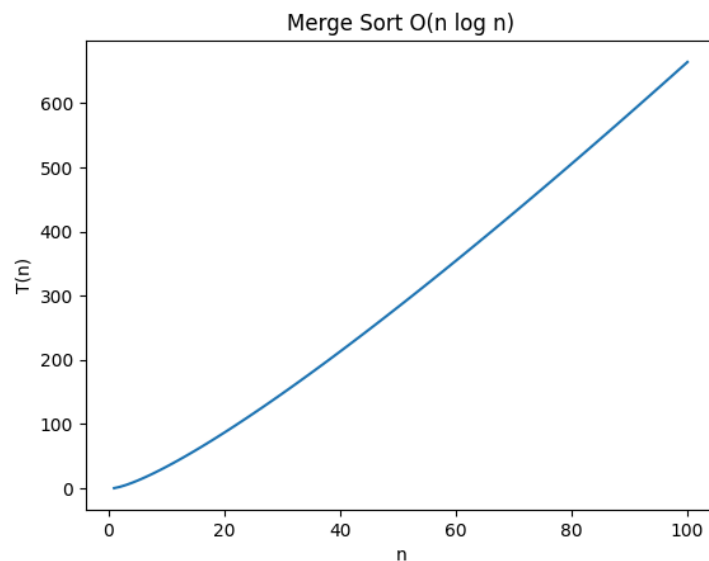
Complejidad:

- Mejor Caso: $O(n \log n)$
- Peor Caso: $O(n \log n)$

Observaciones:

- Parte a la mitad el conjunto para ordenar y luego combinar
- Muy efectivo pero tienen alta recursividad lo que hace que ocupe más memoria de los pasos realizados

Gráfica:



Quick Sort:

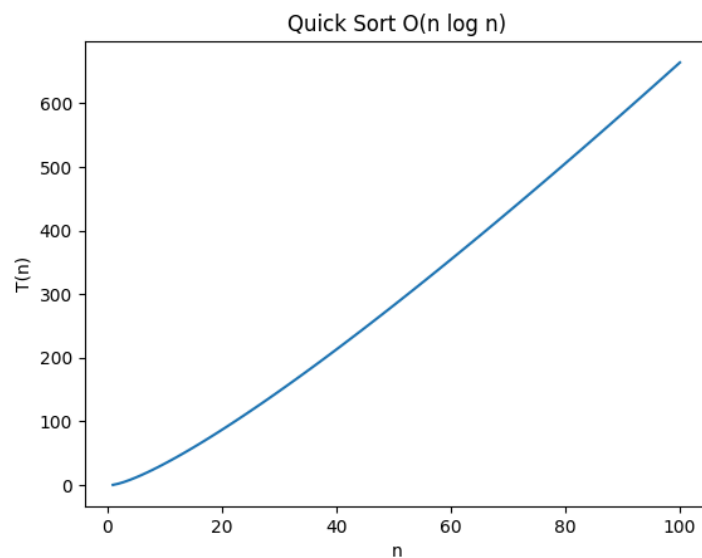
Complejidad:

- Mejor Caso: $O(n \log n)$
- Peor Caso: $O(n^2)$

Observaciones:

- Quick sort funciona seleccionando un pivote con el cual usando de base divide el arreglo que estamos ordenando en menores y mayores
- Más rápido en ejecución que la mayoría de otros métodos

Gráfica:



Random Quick Sort:

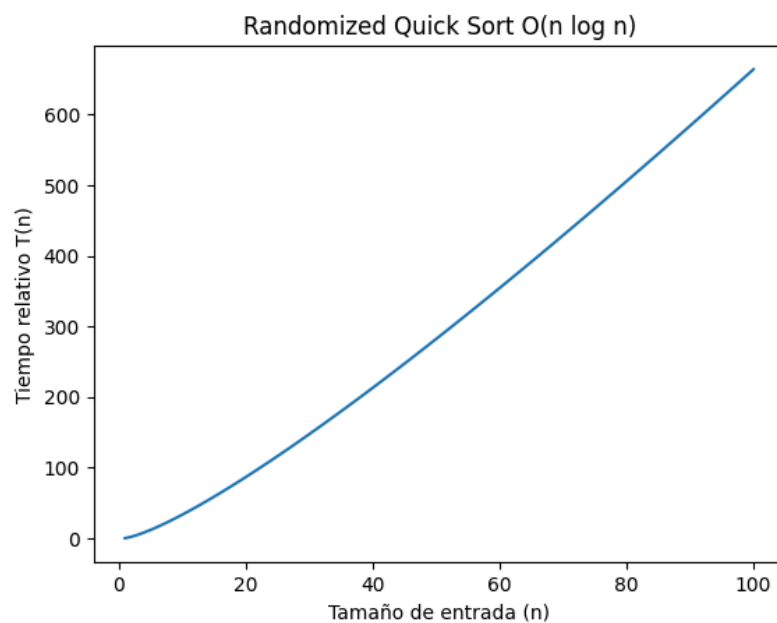
Complejidad:

- Mejor Caso: $O(n \log n)$
- Peor Caso: $O(n^2)$

Observaciones:

- Puede ser describirse como una variante del Quicksort la cual igual selecciona un pivote pero lo hace de manera aleatoria lo que según mis diversos intentos que he hecho reduce mucho la probabilidad del peor caso posible

Gráfico:



Counting Sort:

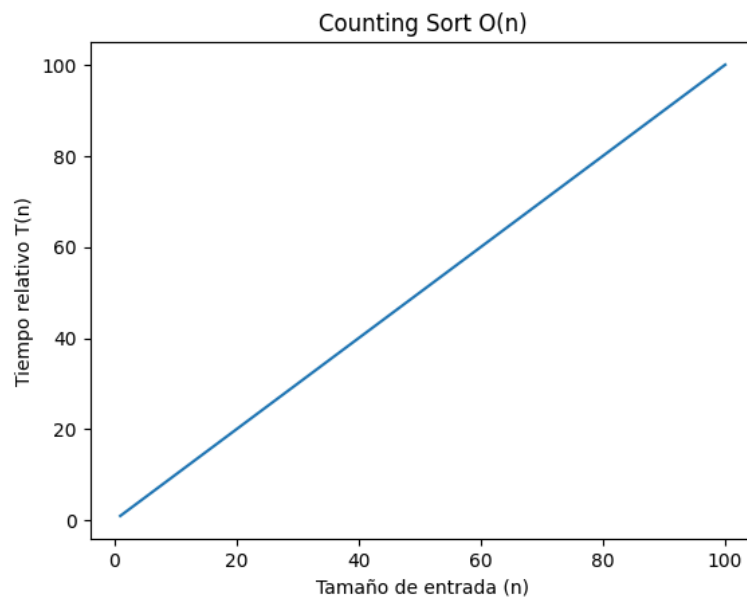
Complejidad:

- Mejor Caso: $O(n + k)$
- Peor Caso: $O(n + k)$

Observaciones:

- Counting sort ordena los elementos dependiendo de cuantas veces haya contado la cantidad de ocurrencias dentro de cada elemento en un rango conocido
- Su efectividad varía según el tamaño de k

Gráfica:



Radix Sort:

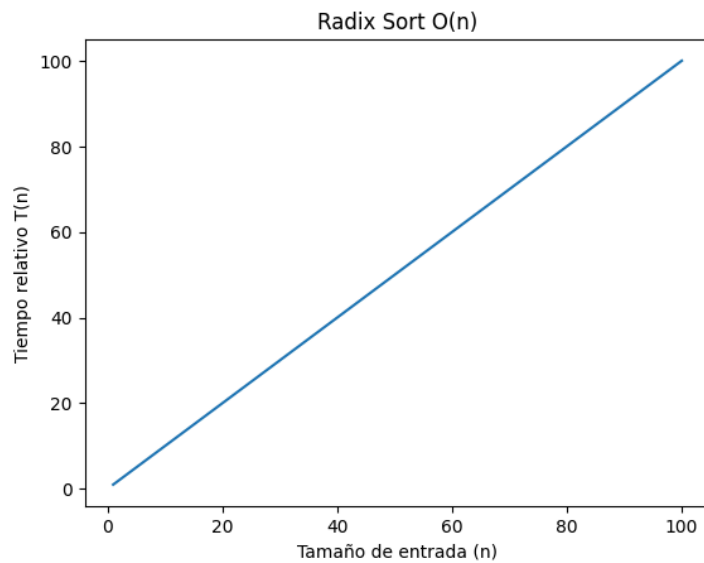
Complejidad:

- Mejor Caso: $O(n * d)$
- Peor Caso: $O(n * d)$

Observaciones:

- Ordena los elementos procesando los dígitos individualmente con una versión de counting sort

Gráfica:



Representaciones en Pseudocodigo:

Bubble Sort:

BUBBLE-SORT(A)

$n \leftarrow \text{longitud}(A)$

PARA $i \leftarrow 0$ HASTA $n-1$

PARA $j \leftarrow 0$ HASTA $n-i-2$

SI $A[j] > A[j+1]$

intercambiar $A[j]$ con $A[j+1]$

Selection Sort:

SELECTION-SORT(A)

$n \leftarrow \text{longitud}(A)$

PARA $i \leftarrow 0$ HASTA $n-1$

$\text{min} \leftarrow i$

PARA $j \leftarrow i+1$ HASTA n

SI $A[j] < A[\text{min}]$

$\text{min} \leftarrow j$

intercambiar $A[i]$ con $A[\text{min}]$

Insertion Sort:

INSERTION-SORT(A)

PARA $i \leftarrow 1$ HASTA $\text{longitud}(A)-1$

$\text{key} \leftarrow A[i]$

$j \leftarrow i - 1$

MIENTRAS $j \geq 0$ Y $A[j] > \text{key}$

$A[j+1] \leftarrow A[j]$

$j \leftarrow j - 1$

$A[j+1] \leftarrow \text{key}$

Merge Sort:

MERGE-SORT(A)

SI longitud(A) ≤ 1

retornar A

dividir A en izquierda y derecha

izquierda $\leftarrow$ MERGE-SORT(izquierda)

derecha $\leftarrow$ MERGE-SORT(derecha)

retornar MERGE(izquierda, derecha)

MERGE(izq, der)

resultado $\leftarrow []$

MIENTRAS izq y der no estén vacíos

SI izq[0] $\leq$ der[0]

agregar izq[0] a resultado

eliminar izq[0]

SINO

agregar der[0] a resultado

eliminar der[0]

agregar lo restante de izq o der

retornar resultado

Quick Sort:

QUICK-SORT(A)

SI longitud(A) ≤ 1

retornar A

pivote $\leftarrow A[0]$

menores $\leftarrow$ elementos $<$ pivote

iguales $\leftarrow$ elementos $=$ pivote

mayores $\leftarrow$ elementos $>$ pivote

retornar QUICK-SORT(menores) + iguales + QUICK-SORT(mayores)

Random Quick Sort:

RANDOMIZED-QUICK-SORT(A)

SI longitud(A) ≤ 1

retornar A

pivote $\leftarrow$ elemento aleatorio de A

menores $\leftarrow$ elementos $<$ pivote

iguales $\leftarrow$ elementos $=$ pivote

mayores $\leftarrow$ elementos $>$ pivote

retornar RANDOMIZED-QUICK-SORT(menores) + iguales +
RANDOMIZED-QUICK-SORT(mayores)

Counting Sort:

COUNTING-SORT(A)

max $\leftarrow$ valor máximo de A

crear arreglo count de tamaño max+1 con ceros

PARA cada elemento x en A

count[x] $\leftarrow$ count[x] + 1

resultado $\leftarrow$ []

PARA i $\leftarrow$ 0 HASTA max

MIENTRAS count[i] $>$ 0

agregar i a resultado

count[i] $\leftarrow$ count[i] - 1

retornar resultado

Radix Sort:

RADIX-SORT(A)

max $\leftarrow$ valor máximo de A

exp $\leftarrow$ 1

MIENTRAS max / exp $>$ 0

A $\leftarrow$ COUNTING-SORT-POR-DIGITO(A, exp)

exp $\leftarrow$ exp * 10

retornar A

COUNTING-SORT-POR-DIGITO(A, exp)

crear $\text{count}[0..9] \leftarrow 0$
 $\text{output} \leftarrow$ arreglo del mismo tamaño

PARA cada elemento x en A
 $\text{índice} \leftarrow (x / \text{exp}) \% 10$
 $\text{count}[\text{índice}]++$

acumular count

construir output en orden

retornar output